

Nginx 反向代理故障排查：499、502、503、504

一、故障概述

错误码	名称	本质原因	简单理解
499	Client Closed Request	客户端主动断开连接	客户端不等了
502	Bad Gateway	无法从后端获取有效响应	后端挂了
503	Service Unavailable	服务器暂时无法处理请求	服务器忙不过来
504	Gateway Timeout	等待后端响应超时	后端太慢了

二、实验环境准备

2.1 创建模拟后端服务

```

# 创建后端服务目录
sudo mkdir -p /var/www/backend-test

# 创建正常响应页面
sudo tee /var/www/backend-test/index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>正常后端</title></head>
<body style="text-align:center;padding:50px;">
<h1>后端服务正常</h1>
</body>
</html>
EOF

# 创建慢响应脚本（用于模拟 504 和 499）
sudo tee /var/www/backend-test/slow.py << 'EOF'

```

```
#!/usr/bin/env python3
import time
from http.server import HTTPServer, SimpleHTTPRequestHandler

class SlowHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        if '/very-slow' in self.path:
            delay = 60
        elif '/slow' in self.path:
            delay = 30
        else:
            delay = 0

        if delay > 0:
            print(f"收到请求, 延迟 {delay} 秒响应...")
            time.sleep(delay)

        self.send_response(200)
        self.send_header('Content-type', 'text/html; charset=utf-8')
        self.end_headers()
        self.wfile.write(f'<h1>响应完成</h1><p>延迟: {delay}秒</p>'.encode())

if __name__ == '__main__':
    server = HTTPServer(('172.16.5.50', 9001), SlowHandler)
    print("后端服务启动在 http://172.16.5.50:9001")
    print("    /           - 立即响应")
    print("    /slow        - 30秒后响应")
    print("    /very-slow   - 60秒后响应")
    server.serve_forever()
EOF

sudo chmod +x /var/www/backend-test/slow.py
```

2.2 创建综合测试配置

```
# 创建故障演示配置
sudo tee /etc/nginx/conf.d/fault-demo.conf << 'EOF'
# ===== Upstream 定义 =====
```

```
# 正常后端
upstream backend_normal {
    server 172.16.5.50:9001;
}

# 不存在的后端 (用于 502)
upstream backend_dead {
    server 172.16.5.50:9999;
}

# 所有后端标记为 down (用于 503)
upstream backend_all_down {
    server 172.16.5.50:9001 down;
    server 172.16.5.50:9002 down;
}

# ===== 限流配置 (用于 503) =====
limit_conn_zone $binary_remote_addr zone=conn_limit:10m;
limit_req_zone $binary_remote_addr zone=req_limit:10m rate=1r/s;

# ===== 服务器配置 =====
server {
    listen 8090;
    server_name 172.16.5.30;

    access_log /var/log/nginx/fault-test.log;
    error_log /var/log/nginx/fault-error.log;

    # ----- 正常访问 -----
    location /normal {
        proxy_pass http://backend_normal/;
        proxy_set_header Host $host;
        proxy_connect_timeout 5s;
        proxy_read_timeout 60s;
    }

    # ----- 触发 502: 后端不存在 -----
    location /test-502 {
        proxy_pass http://backend_dead/;
        proxy_set_header Host $host;
        proxy_connect_timeout 5s;
    }
}
```

```

# ----- 触发 503: 所有后端 down -----
location /test-503-down {
    proxy_pass http://backend_all_down/;
}

# ----- 触发 503: 请求速率限制 -----
location /test-503-rate {
    limit_req zone=req_limit burst=2 nodelay;
    limit_req_status 503;
    proxy_pass http://backend_normal/;
}

# ----- 触发 503: 维护模式 -----
location /maintenance {
    return 503 '{"status":"maintenance","message":"系统维护中"}';
    default_type application/json;
}

# ----- 触发 504: 超时太短 -----
location /test-504 {
    proxy_pass http://backend_normal/slow;
    proxy_set_header Host $host;
    proxy_connect_timeout 5s;
    proxy_read_timeout 3s; # 只等3秒, 后端要30秒
}

# ----- 触发 499: 后端很慢, 客户端会断开 -----
location /test-499 {
    proxy_pass http://backend_normal/very-slow;
    proxy_set_header Host $host;
    proxy_connect_timeout 5s;
    proxy_read_timeout 120s;
}

# ----- 默认页面 -----
location / {
    default_type text/plain;
    return 200 'Fault Demo Server'
}

```

测试路径:

/normal - 正常访问

```
/test-502      - 触发 502 (后端不存在)
/test-503-down - 触发 503 (后端全部down)
/test-503-rate - 触发 503 (速率限制, 快速请求多次)
/maintenance  - 触发 503 (维护模式)
/test-504      - 触发 504 (超时)
/test-499      - 触发 499 (用 curl --max-time 5 测试)

';
    }
}
EOF

# 测试并重载
sudo nginx -t && sudo systemctl reload nginx
```

2.3 启动后端服务

```
# 启动后端
pkill -f "slow.py" 2>/dev/null
cd /var/www/backend-test && python3 slow.py &
sleep 2

# 验证后端
curl -s http://172.16.5.50:9001/ | head -1
```

三、故障一：502 Bad Gateway

3.1 触发 502

```
echo "=== 触发 502: 后端不存在 ==="
curl -i http://172.16.5.30:8090/test-502 2>/dev/null | head -10
```

预期输出：

```
HTTP/1.1 502 Bad Gateway
...
```

3.2 常见原因

原因	说明	检查方法
后端服务未运行	服务崩溃或未启动	<code>systemctl status <service></code>
后端端口错误	proxy_pass 端口配置错误	<code>ss -tlnp grep <port></code>
后端拒绝连接	防火墙或权限问题	<code>curl http://172.16.5.50:<port></code>
后端返回无效响应	应用错误、协议不匹配	查看后端应用日志

3.3 排查步骤

```
# 查看错误日志
echo "=== 错误日志 ==="
sudo tail -3 /var/log/nginx/fault-error.log

# 检查端口监听
echo ""
echo "=== 端口监听状态 ==="
ss -tlnp | grep -E "9001|9999" || echo "端口 9999 未监听"

# 测试后端连通性
echo ""
echo "=== 后端连通性 ==="
curl -s -o /dev/null -w "9001端口: %{http_code}\n"
http://172.16.5.50:9001/ 2>/dev/null || echo "9001: 连接失败"
curl -s -o /dev/null -w "9999端口: %{http_code}\n"
http://172.16.5.50:9999/ 2>/dev/null || echo "9999: 连接失败"
```

3.4 修复方法

```
# 修复：启动后端服务或修正端口配置
# 确保 upstream 中的服务器地址和端口正确
```

四、故障二：504 Gateway Timeout

4.1 触发 504

```
echo "=== 触发 504：后端需要30秒, Nginx只等3秒 ==="
time curl -i http://172.16.5.30:8090/test-504 2>/dev/null | head -5
```

预期输出：

```
HTTP/1.1 504 Gateway Time-out
...
real    0m3.xxx    # 约3秒后超时
```

4.2 常见原因

原因	说明	检查方法
后端处理太慢	数据库慢查询、计算密集	查看后端应用日志
proxy_read_timeout 太短	超时设置不合理	检查 Nginx 配置
网络延迟高	后端服务器网络问题	ping 测试
后端过载	请求积压、资源耗尽	查看后端服务器负载

4.3 排查步骤

```
# 查看错误日志
echo "=== 错误日志（超时相关）==="
sudo grep -i "timed out" /var/log/nginx/fault-error.log | tail -3

# 检查超时配置
echo ""
echo "=== 超时配置 ==="
grep -E "proxy_.*timeout" /etc/nginx/conf.d/fault-demo.conf

# 测试后端响应时间
echo ""
echo "=== 后端响应时间 ==="
timeout 5 curl -s -w "响应时间: %{time_total}s\n"
http://172.16.5.50:9001/ || echo "5秒内无响应"
```

4.4 修复方法

```
# 方法1：增加超时时间（临时方案）
location /api {
    proxy_pass http://backend;
    proxy_connect_timeout 10s;
    proxy_read_timeout 60s;    # 增加读取超时
    proxy_send_timeout 10s;
}

# 方法2：优化后端性能（根本方案）
# - 优化数据库查询
# - 增加缓存
# - 异步处理耗时任务
```

五、故障三：499 Client Closed Request

5.1 什么是 499

499 是 Nginx 特有的状态码。当 Nginx 正在等待后端响应时，客户端主动关闭了连接，Nginx 在日志中记录 499。客户端看不到 499，因为它已经断开了。

5.2 触发 499

```
echo "=== 触发 499：客户端5秒超时，后端需要60秒 ==="
curl --max-time 5 http://172.16.5.30:8090/test-499 2>&1 || echo ""
echo "客户端已超时断开"

sleep 2

echo ""
echo "=== 查看日志中的 499 ==="
sudo grep " 499 " /var/log/nginx/fault-test.log | tail -3
```

5.3 常见原因

原因	说明	典型场景
客户端超时设置短	客户端等不及就断开	移动端网络差
后端处理太慢	响应时间超过客户端预期	复杂查询
用户主动取消	刷新页面或关闭浏览器	页面加载慢
重复提交	用户多次点击提交按钮	表单提交

5.4 排查步骤

```
# 统计 499 数量
echo "=== 499 统计 ==="
sudo grep -c " 499 " /var/log/nginx/fault-test.log 2>/dev/null ||
echo "0"

# 查看 499 请求详情
echo ""
echo "=== 499 请求详情 ==="
sudo grep " 499 " /var/log/nginx/fault-test.log | tail -3
```

5.5 修复方法

```
# 方法1: 忽略客户端断开, 继续处理 (适用于支付等场景)
location /api {
    proxy_pass http://backend;
    proxy_ignore_client_abort on; # 关键配置
}

# 方法2: 优化后端响应时间 (根本方案)
# 方法3: 前端增加超时时间或加载提示
```

六、故障四：503 Service Unavailable

6.1 触发 503

```
# 场景1: 所有后端 down
echo "=== 场景1: 所有后端标记为 down ==="
curl -i http://172.16.5.30:8090/test-503-down 2>/dev/null | head -5
echo ""

# 场景2: 请求速率超限
echo "=== 场景2: 请求速率超限 (快速发送5个请求) ==="
for i in {1..5}; do
    code=$(curl -s -o /dev/null -w "%{http_code}"
http://172.16.5.30:8090/test-503-rate)
    echo "请求 $i: HTTP $code"
done
echo ""

# 场景3: 维护模式
echo "=== 场景3: 维护模式 ==="
curl -s http://172.16.5.30:8090/maintenance
echo ""
```

6.2 常见原因

原因	说明	日志关键词
所有后端不可用	upstream 中服务器全部故障	no live upstreams
连接数超限	limit_conn 限制	limiting connections
请求速率超限	limit_req 限制	limiting requests
主动维护模式	配置了 return 503	无特殊日志

6.3 排查步骤

```
# 查看错误日志
echo "=== 错误日志 ==="
sudo grep -E "(503|no live|limiting)" /var/log/nginx/fault-error.log
| tail -5

# 检查 upstream 配置
echo ""
echo "=== Upstream 中的 down 标记 ==="
grep -E "(upstream|down)" /etc/nginx/conf.d/fault-demo.conf | head
-10

# 检查限流配置
echo ""
echo "=== 限流配置 ==="
grep -E "limit_(conn|req)" /etc/nginx/conf.d/fault-demo.conf | head
-5
```

6.4 修复方法

```
# 方法1: 移除 down 标记或添加可用后端
upstream backend {
    server 172.16.5.50:9001;
    server 172.16.5.50:9002 backup; # 改为 backup 而非 down
}

# 方法2: 调整限流配置
limit_conn conn_zone 100; # 增加连接数限制
limit_req zone=req_zone burst=20 nodelay; # 增加突发请求数

# 方法3: 解除维护模式
# 删除或注释 return 503 配置
```

七、故障对比总结

对比项	499	502	503	504
谁导致	客户端	后端	服务器配置	后端
后端状态	运行中但慢	未运行/崩溃	可能正常	运行中但慢
日志关键词	无特殊	connect() failed	no live / limiting	timed out
排查重点	后端响应时间	后端是否运行	upstream/限流配置	超时配置
快速修复	ignore_client_abort	启动后端	检查配置	增加超时

八、综合排查脚本

```
sudo tee /usr/local/bin/nginx-fault-check.sh << 'EOF'
#!/bin/bash

echo "====="
```

```

echo "Nginx 反向代理故障排查"
echo "===== "

# 1. Nginx 状态
echo ""
echo "[1] Nginx 状态: "
systemctl is-active nginx && echo "✓ 运行中" || echo "✗ 未运行"

# 2. 错误统计
echo ""
echo "[2] 错误统计 (最近1000条日志): "
log_file="/var/log/nginx/access.log"
if [ -f "$log_file" ]; then
    sudo tail -1000 "$log_file" 2>/dev/null | \
        awk '{print $9}' | grep -E "^(499|502|503|504)$" | \
        sort | uniq -c | sort -rn | \
        while read count code; do
            case $code in
                499) desc="Client Closed" ;;
                502) desc="Bad Gateway" ;;
                503) desc="Unavailable" ;;
                504) desc="Timeout" ;;
            esac
            printf "  %s %-15s %s 次\n" "$code" "$desc" "$count"
        done
else
    echo "  日志文件不存在"
fi

# 3. 错误日志关键词
echo ""
echo "[3] 错误日志关键词: "
sudo tail -100 /var/log/nginx/error.log 2>/dev/null | \
    grep -oE "(connect\(\\) failed|timed out|no live
upstreams|limiting)" | \
    sort | uniq -c | sort -rn | head -5 || echo "  无错误"

# 4. 配置检查
echo ""
echo "[4] 配置检查: "
down_count=$(grep -r " down;" /etc/nginx/conf.d/*.conf 2>/dev/null |
wc -l)

```

```

echo "  标记为 down 的服务器: $down_count"
limit_count=$(grep -rE "limit_(conn|req) " /etc/nginx/conf.d/*.conf
2>/dev/null | wc -l)
echo "  限流配置: $limit_count 处"

# 5. 后端端口检查
echo ""
echo "[5] 常见后端端口: "
for port in 9001 9002 9003 8080 3000 5000; do
    if ss -tlnp 2>/dev/null | grep -q ":$port "; then
        echo "  端口 $port: ✓"
    fi
done

echo ""
echo "===== "
echo "排查建议: "
echo "  499 → 检查后端响应时间"
echo "  502 → 检查后端是否运行"
echo "  503 → 检查 upstream 和限流配置"
echo "  504 → 增加超时或优化后端"
echo "===== "
EOF

sudo chmod +x /usr/local/bin/nginx-fault-check.sh

# 运行
sudo /usr/local/bin/nginx-fault-check.sh

```

九、常用配置模板

9.1 超时配置

```
# 一般业务
proxy_connect_timeout 5s;
proxy_read_timeout 60s;
proxy_send_timeout 10s;

# 长连接业务（文件上传等）
proxy_connect_timeout 10s;
proxy_read_timeout 300s;
proxy_send_timeout 300s;
```

9.2 健康检查与故障转移

```
upstream backend {
    server 172.16.5.50:9001 max_fails=3 fail_timeout=30s;
    server 172.16.5.50:9002 max_fails=3 fail_timeout=30s;
    server 172.16.5.50:9003 backup;
}

location / {
    proxy_pass http://backend;
    proxy_next_upstream error timeout http_502 http_503 http_504;
    proxy_next_upstream_tries 2;
}
```

9.3 限流配置

```
limit_conn_zone $binary_remote_addr zone=conn:10m;
limit_req_zone $binary_remote_addr zone=req:10m rate=10r/s;

location /api {
    limit_conn conn 100;
    limit_req zone=req burst=20 nodelay;
    proxy_pass http://backend;
}
```

十、清理实验环境

```
# 停止后端服务
kill -f "slow.py"

# 查看实验配置
ls /etc/nginx/conf.d/fault-*.conf 2>/dev/null

# 如需删除
# sudo rm /etc/nginx/conf.d/fault-demo.conf
# sudo rm /var/log/nginx/fault-*.log
# sudo systemctl reload nginx
```

十一、快速参考卡片

故障快速判断

499 → 客户端断开 → 后端太慢，客户端不等了

502 → 连接失败 → 后端没运行或端口错误

503 → 服务不可用 → upstream 全挂或触发限流

504 → 响应超时 → 后端太慢，超过 Nginx 等待时间

常用排查命令

sudo tail -f /var/log/nginx/error.log # 查看错误

ss -tlnp | grep <port> # 检查端口

curl -v http://172.16.5.50:<port>/ # 测试后端

grep "proxy_.*timeout" /etc/nginx/conf.d/* # 超时配置